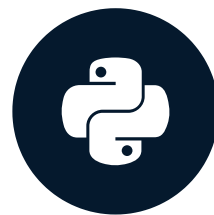


Built-in functions

INTERMEDIATE PYTHON FOR DEVELOPERS



Jasmin Ludolf

Senior Data Science Content Developer

What we'll cover

- **Chapter 1:**
 - Built-in functions, modules, and packages
- **Chapter 2 :**
 - Custom functions
- **Chapter 3:**
 - Error-handling

Introduction to Python for Developers

Functions we know

```
# Printing  
print("Password is accepted!")
```

```
Password is accepted!
```

```
# Checking data types  
type(print)
```

```
builtin_function_or_method
```

```
# Password attempts  
for attempt in range(1, 4):  
    print("Attempt", attempt)
```

```
Attempt 1  
Attempt 2  
Attempt 3
```

Built-in functions

- Help build features with less code
- Performance monitoring dashboard ☐



max() and min()

```
# List of preparation times (minutes)
preparation_times = [19.23, 15.67, 48.57, 23.45, 12.06, 34.56, 45.67]

# Find the longest preparation time
print(max(preparation_times))
```

48.57

```
# Find the shortest preparation time
print(min(preparation_times))
```

12.06

sum() and round()

```
# Calculate the total preparation time  
print(sum(preparation_times))
```

199.21

```
# Store total time  
total_time = sum(preparation_times)  
  
# Round to one decimal place  
print(round(total_time, 1))
```

199.2

len()

- Counts the number of elements

```
# Count the number of orders  
print(len(preparation_times))
```

```
7
```

```
# Calculate average preparation time  
print(sum(preparation_times) / len(preparation_times))
```

```
28.4585714
```


len()

- Counts the number of characters, including spaces

```
# Length of a string  
print(len("Burger Hub"))
```

```
10
```


sorted()

```
# Sort a list in ascending order  
print(sorted(preparation_times))
```

```
[12.06, 15.67, 19.23, 23.45, 34.56,  
45.67, 48.57]
```

```
# Sort a string alphabetically  
print(sorted("George"))
```

```
['G', 'e', 'e', 'g', 'o', 'r']
```

Benefits of functions

- Perform complex tasks with less code

```
# Find total preparation time  
print(sum(preparation_times))
```

```
199.21
```

Benefits of functions

```
# Find total preparation time
# Create a variable to increment
time_count = 0

# Loop through preparation times
for time in preparation_times:
    # Add each time to time_count
    time_count += time
    print(time_count)
```

- `sum()` is reusable, shorter, cleaner, and less prone to errors!

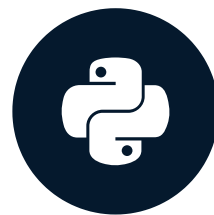
```
19.23
34.9
83.47
106.92
118.98
153.54
199.21
```

Let's practice!

INTERMEDIATE PYTHON FOR DEVELOPERS

Modules

INTERMEDIATE PYTHON FOR DEVELOPERS



Jasmin Ludolf

Senior Data Science Content Developer

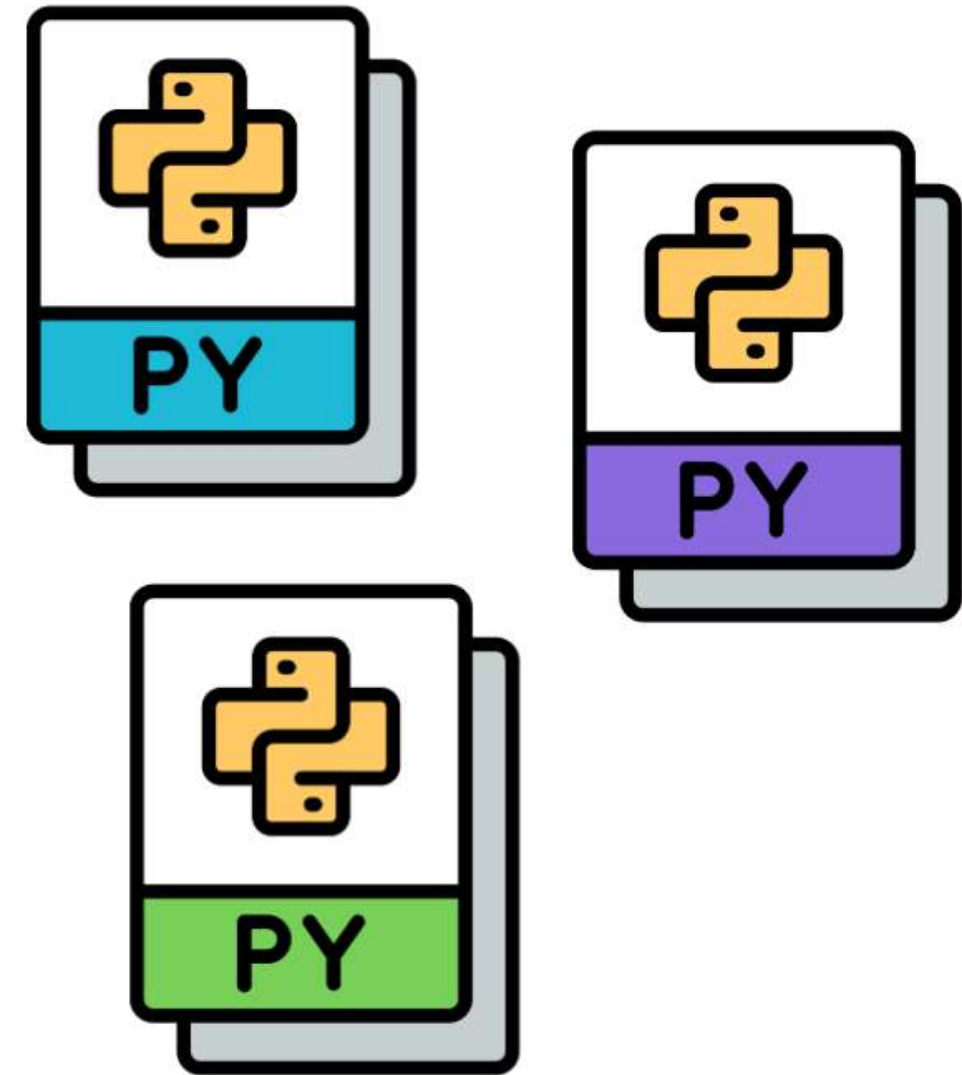
What are modules?

- Modules are Python scripts
 - Files ending with `.py`
 - Contain functions and attributes
 - Can contain other modules
- Help us avoid rewriting code that already exists!



Python modules

- There are around 200 built-in modules
 - [Documentation](#)
- `os` :
 - Used for interacting with our operating system
 - Check the current directory
 - List available files
 - Access environment variables
- `string` :
 - Simplifies text processing tasks



Importing a module

```
# General syntax  
import <module_name>
```

```
# Import the os module  
import os
```

```
# Check the type  
print(type(os))
```

```
<class 'module'>
```


Finding a module's functions

- Check the [documentation](#)

```
# Call help()
# Warning - will return a very large output!
print(help(os))
```

Help on module os:

NAME

os - OS routines for NT or Posix depending on what system we're on.

MODULE REFERENCE

<https://docs.python.org/3.12/library/os.html>

...

Getting the current working directory

```
# Using an os function  
print(os.getcwd())
```

```
/home/courses/intermediate_python_for_developers
```

- Useful if we need to reference the directory later

```
# Assign to a variable  
work_dir = os.getcwd()
```

Changing directory

```
# Changing directory  
os.chdir("/home/courses")
```

```
# Check the current directory  
print(os.getcwd())
```

```
/home/courses
```

```
# Confirm work_dir has not changed  
print(work_dir)
```

```
/home/courses/intermediate_python_for_developers
```

Module attributes

- Attributes return values
- Functions perform tasks
- Don't use parentheses with attributes

```
# Get the local environment  
print(os.environ)
```

```
environ{'PATH': '/usr/local/bin',  
        'TERM': 'xterm',  
        'HOSTNAME': '097a0fe4-d6ce-4325-a6e2-1d0ce2800c2b',  
        'TZ': 'Europe/Brussels',  
        'LANG': 'en_US.UTF-8',  
        ...}
```

String module

```
import string
```

```
print(string.ascii_lowercase)
```

```
abcdefghijklmnopqrstuvwxyz
```

- Check if a string contains letters, numbers, or specific characters
- Handy for validating user input ☐

```
print(string.digits)
```

```
0123456789
```

```
print(string.punctuation)
```

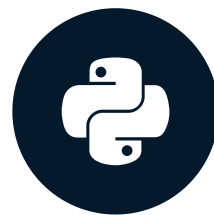
```
!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~
```

Let's practice!

INTERMEDIATE PYTHON FOR DEVELOPERS

Packages

INTERMEDIATE PYTHON FOR DEVELOPERS



Jasmin Ludolf

Senior Data Science Content Developer

Modules are Python files

- Module = Python file
- Anyone can create a Python file!



Packages

- A collection of modules = **Package**
 - Also called a **library**
- Publicly available and free
- Downloaded from PyPI
- Then can be imported and used like modules



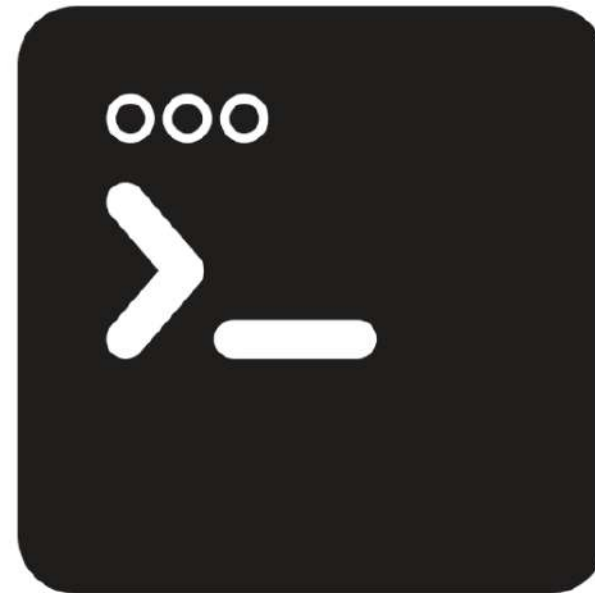
¹ <https://pypi.org/>

Installing a package

- Terminal / Command Prompt

```
python3 -m pip install <package_name>
```

- `python3` - executes Python code from the terminal
- `pip` - preferred installer



Installing a package

```
python3 -m pip install pandas
```



- Package for data manipulation and analysis

Importing with an alias

```
# Import pandas  
import pandas
```

- Use an alias to shorten the code

```
# Import pandas using an alias  
import pandas as pd
```

Creating a DataFrame

```
# Sales dictionary
sales = {"user_id": ["KM37", "PR19", "YU88"],
        "order_value": [197.75, 208.21, 134.99]}

# Convert to a pandas DataFrame
sales_df = pd.DataFrame(sales)

print(sales_df)
```

	user_id	order_value
0	KM37	197.75
1	PR19	208.21
2	YU88	134.99

Reading in a CSV file

```
# Reading in a CSV file in our current directory
sales_df = pd.read_csv("sales.csv")

# Checking the data type
print(type(sales_df))
```

```
pandas.core.frame.DataFrame
```

Previewing the file

```
# DataFrame method to preview the first five rows  
print(sales_df.head())
```

	user_id	order_value
0	KM37	197.75
1	PR19	208.21
2	YU88	134.99
3	NT43	153.54
4	IW06	379.47

Checking the file info

```
# Checking the file info
print(sales_df.info())
```

```
RangeIndex: 3 entries, 0 to 2
Data columns (total 2 columns):
 #   Column          Non-Null Count  Dtype
<hr />  -----  -
 0   user_id         3 non-null     object
 1   order_value     3 non-null     float64
dtypes: float64(1), object(1)
memory usage: 180.0+ bytes
```


Functions versus methods

```
# This is a built-in function  
print(sum([1, 2, 3, 4, 5]))
```

15

- Function = code to perform a task

```
# This is a pandas function  
sales_df = pd.DataFrame(sales)
```

- `.head()` only works with pandas DataFrames

```
# This is a method  
print(sales_df.head())
```

	user_id	order_value
0	KM37	197.75
1	PR19	208.21
2	YU88	134.99
3	NT43	153.54
4	IW06	379.47

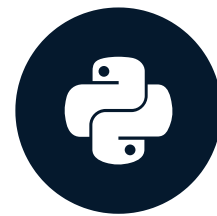
- Method = a function that is specific to a data type

Let's practice!

INTERMEDIATE PYTHON FOR DEVELOPERS

Defining a custom function

INTERMEDIATE PYTHON FOR DEVELOPERS



Jasmin Ludolf

Senior Data Science Content Developer

Calculating the average

```
# List of preparation times (minutes)
preparation_times = [19.23, 15.67, 48.57, 23.45, 12.06, 34.56, 45.67]

# Calculating average preparation time
average_time = sum(preparation_times) / len(preparation_times)

# Rounding the results
rounded_average_time = round(average_time, 2)
print(average_time)
```

28.46

When to make a custom function

Don't Repeat Yourself (DRY)

- Considerations for making a custom function:
 - Number of lines
 - Code complexity
 - Frequency of use



Creating a custom function

```
# Create a custom function to calculate the average value  
def
```

Creating a custom function

```
# Create a custom function to calculate the average value  
def average
```


Creating a custom function

```
# Create a custom function to calculate the average value  
def average(
```


Creating a custom function

```
# Create a custom function to calculate the average value  
def average(values)
```

- `values` (argument) - information the function needs to do its job

Creating a custom function

```
# Create a custom function to calculate the average value  
def average(values):
```

Creating a custom function

```
# Create a custom function to calculate the average value
def average(values):
    # Calculate the average
    average_value = sum(values) / len(values)
```

Creating a custom function

```
# Create a custom function to calculate the average value
def average(values):
    # Calculate the average
    average_value = sum(values) / len(values)

    # Round the results
    rounded_average = round(average_value, 2)
```

Creating a custom function

```
# Create a custom function to calculate the average value
def average(values):
    # Calculate the average
    average_value = sum(values) / len(values)

    # Round the results
    rounded_average = round(average_value, 2)

    # Return an output
    return
```

- `average_value` and `rounded_average` are only available within `average()`

Creating a custom function

```
# Create a custom function to calculate the average value
def average(values):
    # Calculate the average
    average_value = sum(values) / len(values)

    # Round the results
    rounded_average = round(average_value, 2)

    # Return rounded_average as an output
    return rounded_average
```

- Documentation is essential □

Using a custom function

```
# List of preparation times (minutes)
preparation_times = [19.23, 15.67, 48.57, 23.45, 12.06, 34.56, 45.67]

# Calculating the average
print(average(preparation_times))
```

28.46

```
# List of orders
orders = [12, 8, 10, 9, 15, 21, 16]
print(average(orders))
```

12.86

Storing a function's output

```
# Calculating the average  
print(average(preparation_times))
```

28.46

```
# Storing average_time  
average_time = average(preparation_times)  
print(average_time)
```

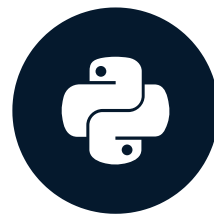
28.46

Let's practice!

INTERMEDIATE PYTHON FOR DEVELOPERS

Default and keyword arguments

INTERMEDIATE PYTHON FOR DEVELOPERS



Jasmin Ludolf

Senior Data Science Content Developer

Average

```
# Create a custom function
def average(values):
    # Calculate the average
    average_value = sum(values) / len(values)

    # Round the results
    rounded_average = round(average_value, 2)

    # Return rounded_average as an output
    return rounded_average
```

- `values` = Argument

Arguments

- Values provided to a function or method
 - **Positional**
 - **Keyword**

Positional arguments

- Provide arguments in order, separated by commas

```
# Round pi to 2 digits  
print(round(3.1415926535, 2))
```

```
3.14
```

Keyword arguments

- Provide arguments by assigning values to keywords
- Useful for interpretation and tracking arguments

```
# Round pi to 2 digits  
print(round(number=3.1415926535
```

Keyword arguments

- Provide arguments by assigning values to `keywords`
- Useful for interpretation and tracking arguments

```
# Round pi to 2 digits  
print(round(number=3.1415926535, ndigits=2))
```

```
3.14
```


Identifying keyword arguments

```
# Get more information about the help function  
print(help(round))
```

Help on built-in function round in module builtins:

```
round(number, ndigits=None)
```

Round a number to a given precision in decimal digits.

The return value is an integer if ndigits is omitted or None. Otherwise, the return value has the same type as the number. ndigits may be negative.

Keyword arguments

Help on built-in function round in module builtins:

```
round(number, ndigits=None)
```

Round a number to a given precision in decimal digits.

The return value is an integer if ndigits is omitted or None. Otherwise the return value has the same type as the number. ndigits may be negative.

- First argument: `number`
- Second argument: `ndigits`

Default arguments

Help on built-in function round in module builtins:

```
round(number, ndigits=None)
```

Round a number to a given precision in decimal digits.

The return value is an integer if ndigits is omitted or None. Otherwise, the return value has the same type as the number. ndigits may be negative.

- `None` = no value / empty
- Default argument: way of setting a `default` value for an `argument`
- We overwrite `None` to `2`
- Commonly used value - set it using a default argument

Adding an argument

```
# Create a custom function
def average(values):
    average_value = sum(values) / len(values)
    rounded_average = round(average_value, 2)
    return rounded_average
```

Adding an argument

```
# Create a custom function  
def average(values, rounded=False):
```

Adding an argument

```
# Create a custom function
def average(values, rounded=False):
    # Round average to two decimal places if rounded is True
    if rounded == True:
        average_value = sum(values) / len(values)
        rounded_average = round(average_value, 2)
    return rounded_average
```

Adding an argument

```
# Create a custom function
def average(values, rounded=False):
    # Round average to two decimal places if rounded is True
    if rounded == True:
        average_value = sum(values) / len(values)
        rounded_average = round(average_value, 2)
        return rounded_average
    # Otherwise, don't round
    else:
        average_value = sum(values) / len(values)
        return average_value
```

Using the modified `average()` function

```
# List of preparation times (minutes)
preparation_times = [19.23, 15.67, 48.57, 23.45, 12.06, 34.56, 45.67]
```


Using the modified average() function

```
# Get the average without rounding  
print(average(preparation_times, False))
```

28.4585714

```
# Get the average without rounding  
print(average(preparation_times))
```

28.4585714

Using the modified `average()` function

```
# Get the rounded average  
print(average(values=preparation_times, rounded=True))
```

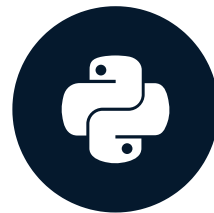
```
28.46
```

Let's practice!

INTERMEDIATE PYTHON FOR DEVELOPERS

Docstrings

INTERMEDIATE PYTHON FOR DEVELOPERS



Jasmin Ludolf

Senior Data Science Content Developer

Docstrings

- String (block of text) describing a function
- Help users understand how to use a function

```
Help on built-in function round in module builtins:
```

```
round(number, ndigits=None)
```

```
    Round a number to a given precision in decimal digits.
```

```
    The return value is an integer if ndigits is omitted or None. Otherwise  
    the return value has the same type as the number. ndigits may be negative.
```

Accessing a docstring

```
# Access information, including the docstring  
print(help(round))
```

Help on built-in function round in module builtins:

```
round(number, ndigits=None)
```

Round a number to a given precision in decimal digits.

The return value is an integer if ndigits is omitted or None. Otherwise the return value has the same type as the number. ndigits may be negative.

None

Accessing a docstring

```
# Access only the docstring  
print(round
```

Accessing a docstring

```
# Access only the docstring  
print(round.
```


Accessing a docstring

```
# Access only the docstring  
print(round.__doc__)
```


Accessing a docstring

```
# Access only the docstring  
print(round.__doc__)
```

Accessing a docstring

```
# Access only the docstring  
print(round.__doc__)
```

Round a number to a given precision in decimal digits.

The return value is an integer if ndigits is omitted or None. Otherwise the return value has the same type as the number. ndigits may be negative.

- `.__doc__` : "dunder-doc" attribute

Creating a docstring

```
def average(values):  
    # One-line docstring  
    """Find the mean in a sequence of values and round to two decimal places."""  
    average_value = sum(values) / len(values)  
    rounded_average = round(average_value, 2)  
    return rounded_average
```

Accessing the docstring

```
# Access our docstring  
print(average.__doc__)
```

Find the mean in a sequence of values and round to two decimal places.

Updating a docstring

```
# Update a function's docstring
average.__doc__ = "Calculate the mean of values in a data structure, rounding the results to 2 digits."

print(help(average))
```

```
Help on function average in module __main__:
```

```
average(values)
```

```
    Calculate the mean of values in a data structure, rounding the results to 2 digits.
```

Multi-line docstring

```
def average(values):  
    """  
    Find the mean in a sequence of values and round to two decimal places.  
  
    """  
    average_value = sum(values) / len(values)  
    rounded_average = round(average_value, 2)  
    return rounded_average
```

Multi-line docstring

```
def average(values):  
    """  
    Find the mean in a sequence of values and round to two decimal places.  
  
    Args:  
  
    """  
    average_value = sum(values) / len(values)  
    rounded_average = round(average_value, 2)  
    return rounded_average
```


Multi-line docstring

```
def average(values):  
    """  
    Find the mean in a sequence of values and round to two decimal places.  
  
    Args:  
        values (list): A list of numeric values.  
  
    """  
    average_value = sum(values) / len(values)  
    rounded_average = round(average_value, 2)  
    return rounded_average
```


Multi-line docstring

```
def average(values):  
    """  
    Find the mean in a sequence of values and round to two decimal places.  
  
    Args:  
        values (list): A list of numeric values.  
  
    Returns:  
        rounded_average (float): The mean of values, rounded to two decimal places.  
    """  
    average_value = sum(values) / len(values)  
    rounded_average = round(average_value, 2)  
    return rounded_average
```

Accessing the docstring

```
# Help  
print(help(average))
```

```
Help on function average in module __main__:
```

```
average(values)
```

```
    Find the mean in a sequence of values and round to two decimal places.
```

```
    Args:
```

```
        values (list): A list of numeric values.
```

```
    Returns:
```

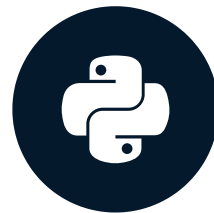
```
        rounded_average (float): The mean of values, rounded to two decimal places.
```

Let's practice!

INTERMEDIATE PYTHON FOR DEVELOPERS

Arbitrary arguments

INTERMEDIATE PYTHON FOR DEVELOPERS



Jasmin Ludolf
Curriculum Manager

Limitations of defined arguments

```
def average(values):  
    """Find the mean in a sequence of values and round to two decimal places."""  
  
    average_value = sum(values) / len(values)  
    rounded_average = round(average_value, 2)  
    return rounded_average  
  
# Using six arguments  
print(average(15, 29, 4, 13, 11, 8))
```

`TypeError: average() takes 1 positional argument but 6 were given`

Arbitrary positional arguments

- Docstrings help clarify how to use custom functions
- Arbitrary arguments allow functions to accept **any number** of arguments

```
# Allow any number of positional, non-keyword arguments
def average(*args):
    # Function code remains the same
```

- Conventional naming: `*args`
- Allows a variety of uses while producing expected results!

Using arbitrary positional arguments

```
# Calling average with six positional arguments  
print(average(15, 29, 4, 13, 11, 8))
```

13.33

Args create a single iterable

- `*` : Convert arguments to a single iterable (tuple)

```
# Calculating across multiple lists  
print(average(*[15, 29], *[4, 13], *[11, 8]))
```

```
13.33
```


Arbitrary keyword arguments

```
# Use arbitrary keyword arguments
def average(**kwargs):
    average_value = sum(kwargs.values()) / len(kwargs.values())
    rounded_average = round(average_value, 2)
    return rounded_average
```

- Arbitrary keyword arguments: `**kwargs`
- `keyword=value`

Using arbitrary keyword arguments

```
# Calling average with six kwargs  
print(average(a=15, b=29, c=4, d=13, e=11, f=8))
```

13.33

```
# Calling average with one kwarg  
print(average(**{"a":15, "b":29, "c":4, "d":13, "e":11, "f":8}))
```

13.33

- Each key-value pair in the dictionary is mapped to a keyword argument and value!

Kwargs create a single iterable

```
# Calling average with three kwargs  
print(average(**{"a":15, "b":29}, **{"c":4, "d":13}, **{"e":11, "f":8}))
```

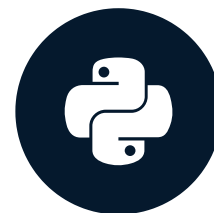
13.33

Let's practice!

INTERMEDIATE PYTHON FOR DEVELOPERS

Lambda functions

INTERMEDIATE PYTHON FOR DEVELOPERS



Jasmin Ludolf

Senior Data Science Content Developer

Simple functions

```
def average(values):  
    average_value = sum(values) / len(values)  
    return average_value
```

Lambda functions

- `lambda` keyword
 - Represents an *anonymous function*

`lambda`

Lambda functions

- `lambda` keyword
 - Represents an *anonymous function*

`lambda` arguments

Lambda functions

- `lambda` keyword
 - Represents an *anonymous function*

`lambda` arguments:

Lambda functions

- `lambda` keyword
 - Represents an *anonymous function*

`lambda arguments: expression`

- Convention is to use `x` for a single argument
- The `expression` is the equivalent of the function body
- No `return` statement is required
- Can be stored as a variable

Creating a lambda function

```
# Lambda average function  
print(lambda x: sum(x) / len(x))
```

```
<function <lambda> at 0x7f11ab813d80>
```

```
# Custom average function  
def average(x):  
    return sum(x) / len(x)  
  
print(average)
```

```
<function average at 0x7f11ab813ec0>
```

Using lambda functions

```
# Get the average  
(lambda x: sum(x) / len(x))
```

Using lambda functions

```
# Get the average  
(lambda x: sum(x) / len(x))([3, 6, 9])
```

```
# Print the average  
print((lambda x: sum(x) / len(x))([3, 6, 9]))
```

```
6.0
```

Storing and calling a lambda function

```
# Store lambda function as a variable
average = lambda x: sum(x) / len(x)

# Call the average function
print(average([3, 6, 9]))
```

```
6.0
```

Multiple parameters

```
# Lambda function with two arguments  
power = lambda x, y: x**y  
  
# Raise 2 to the power of 3  
print(power(2, 3))
```

8

Lambda functions with iterables

- `map()` applies a function to **all** elements in an iterable

```
names = ["john", "sally", "leah"]  
# Apply a lambda function inside map()  
capitalize = map(lambda x: x.capitalize(), names)  
print(capitalize)
```

```
<map object at 0x7fb200529c10>
```

```
# Convert to a list  
print(list(capitalize))
```

```
['John', 'Sally', 'Leah']
```


Custom vs. lambda functions

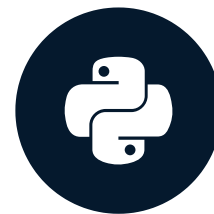
Scenario	Function Type
Complex task	Custom
Same task several times	Custom
Performed once	Lambda
Simple task	Lambda

Let's practice!

INTERMEDIATE PYTHON FOR DEVELOPERS

Introduction to errors

INTERMEDIATE PYTHON FOR DEVELOPERS

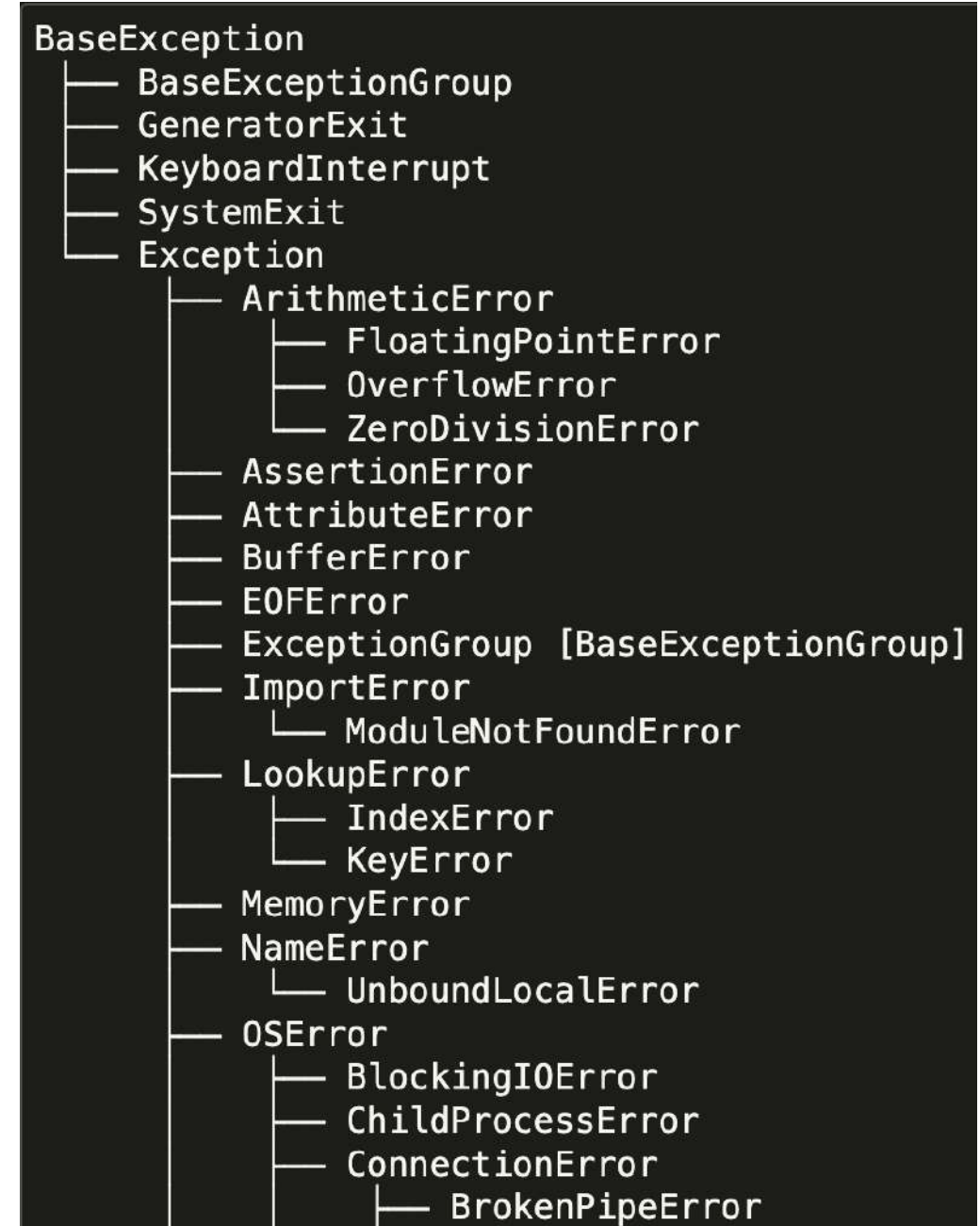


Jasmin Ludolf

Senior Data Science Content Developer

What is an error?

- Code that violates a rule
- Error = Exception
- Cause our program to terminate!



¹ <https://docs.python.org/3/library/exceptions.html#exception-hierarchy>

TypeError

- Incompatible data type

```
"Hello" + 5
```

```
-----  
TypeError
```

```
Traceback (most recent call last)
```

```
Cell In[1], line 1
```

```
----> 1 "Hello" + 5
```

```
TypeError: can only concatenate str (not "int") to str
```

ValueError

```
float("Hello")
```

```
-----  
ValueError                                Traceback (most recent call last)  
Cell In[2], line 1  
----> 1 float("Hello")  
  
ValueError: could not convert string to float: 'Hello'
```

```
float("2")
```

```
2.0
```

- The value is not within an acceptable range

Tracebacks

ValueError

Cell In[2], line 1

----> 1 float("Hello")

ValueError: could not convert string to float: 'Hello'

Traceback (most recent call last)



Tracebacks

```
-----  
ValueError                                Traceback (most recent call last)  
Cell In[2], line 1  
----> 1 float("Hello")  
  
ValueError: could not convert string to float: 'Hello'
```



Tracebacks

ValueError

Traceback (most recent call last)

Cell In[2], line 1

----> 1 float("Hello")

ValueError: could not convert string to float: 'Hello'

Code in packages

- Packages contain other people's code, e.g., custom functions
- Known as source code
- `pip install <package>` downloads source code to our local environment
- The pandas' `pd.read_csv()` function executes the code written for that custom function behind the scenes

Tracebacks from packages

```
# Import pandas package
import pandas as pd

# Create pandas DataFrame
products = pd.DataFrame({"ID": "ABC1",
                        "price": 29.99})

# Try to access the non-existent "tag" column
products["tag"]
```

Tracebacks from packages

```
-----
KeyError                                Traceback (most recent call last)
File /usr/local/lib/python3.8/dist-packages/pandas/core/indexes/base.py:3803, in Index.get_loc(self, key, method, tolerance)
    3802 try:
-> 3803     return self._engine.get_loc(casted_key)
    3804 except KeyError as err:

File /usr/local/lib/python3.8/dist-packages/pandas/_libs/index.pyx:138, in pandas._libs.index.IndexEngine.get_loc()

File /usr/local/lib/python3.8/dist-packages/pandas/_libs/index.pyx:165, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/hashtable_class_helper.pxi:5745, in pandas._libs.hashtable.PyObjectHashTable.get_item()

File pandas/_libs/hashtable_class_helper.pxi:5753, in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'tag'

The above exception was the direct cause of the following exception:

KeyError                                Traceback (most recent call last)
Cell In[5], line 9
      5 products = pd.DataFrame({"ID": ["ABC1", "ABC2", "ABC3"],
      6                        "price": [29.99, 39.95, 51.25]})
      8 # Try to access the non-existent "tag" column
----> 9 products["tag"]

File /usr/local/lib/python3.8/dist-packages/pandas/core/frame.py:3804, in DataFrame.__getitem__(self, key)
    3802 if self.columns.nlevels > 1:
    3803     return self._getitem_multilevel(key)
-> 3804 indexer = self.columns.get_loc(key)
    3805 if is_integer(indexer):
    3806     indexer = [indexer]

File /usr/local/lib/python3.8/dist-packages/pandas/core/indexes/base.py:3805, in Index.get_loc(self, key, method, tolerance)
    3803     return self._engine.get_loc(casted_key)
    3804 except KeyError as err:
-> 3805     raise KeyError(key) from err
    3806 except TypeError:
    3807     # If we have a listlike key, _check_indexing_error will raise
    3808     # InvalidIndexError. Otherwise we fall through and re-raise
    3809     # the TypeError.
    3810     self._check_indexing_error(key)

KeyError: 'tag'
```

Tracebacks from packages

```
-----
KeyError                                Traceback (most recent call last)
File /usr/local/lib/python3.8/dist-packages/pandas/core/indexes/base.py:3803, in Index.get_loc(self, key, method, tolerance)
    3802 try:
-> 3803     return self._engine.get_loc(casted_key)
    3804 except KeyError as err:

File /usr/local/lib/python3.8/dist-packages/pandas/_libs/index.pyx:138, in pandas._libs.index.IndexEngine.get_loc()

File /usr/local/lib/python3.8/dist-packages/pandas/_libs/index.pyx:165, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/hashtable_class_helper.pxi:5745, in pandas._libs.hashtable.PyObjectHashTable.get_item()

File pandas/_libs/hashtable_class_helper.pxi:5753, in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'tag'

The above exception was the direct cause of the following exception:
```


Tracebacks from packages

```
-----
KeyError                                Traceback (most recent call last)
File /usr/local/lib/python3.8/dist-packages/pandas/core/indexes/base.py:3803, in Index.get_loc(self, key, method, tolerance)
    3802 try:
-> 3803     return self._engine.get_loc(casted_key)
    3804 except KeyError as err:

File /usr/local/lib/python3.8/dist-packages/pandas/_libs/index.pyx:138, in pandas._libs.index.IndexEngine.get_loc()

File /usr/local/lib/python3.8/dist-packages/pandas/_libs/index.pyx:165, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/hashtable_class_helper.pxi:5745, in pandas._libs.hashtable.PyObjectHashTable.get_item()

File pandas/_libs/hashtable_class_helper.pxi:5753, in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'tag'

The above exception was the direct cause of the following exception:
```

Tracebacks from packages

```
-----
KeyError                                Traceback (most recent call last)
File /usr/local/lib/python3.8/dist-packages/pandas/core/indexes/base.py:3803, in Index.get_loc(self, key, method, tolerance)
    3802 try:
-> 3803     return self._engine.get_loc(casted_key)
    3804 except KeyError as err:

File /usr/local/lib/python3.8/dist-packages/pandas/_libs/index.pyx:138, in pandas._libs.index.IndexEngine.get_loc()

File /usr/local/lib/python3.8/dist-packages/pandas/_libs/index.pyx:165, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/hashtable_class_helper.pxi:5745, in pandas._libs.hashtable.PyObjectHashTable.get_item()

File pandas/_libs/hashtable_class_helper.pxi:5753, in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'tag'

The above exception was the direct cause of the following exception:
```

Tracebacks from packages

```
-----  
KeyError                                Traceback (most recent call last)  
File /usr/local/lib/python3.8/dist-packages/pandas/core/indexes/base.py:3803, in Index.get_loc(self, key, method, tolerance)  
    3802 try:  
-> 3803     return self._engine.get_loc(casted_key)  
    3804 except KeyError as err:  
  
File /usr/local/lib/python3.8/dist-packages/pandas/_libs/index.pyx:138, in pandas._libs.index.IndexEngine.get_loc()  
  
File /usr/local/lib/python3.8/dist-packages/pandas/_libs/index.pyx:165, in pandas._libs.index.IndexEngine.get_loc()  
  
File pandas/_libs/hashtable_class_helper.pxi:5745, in pandas._libs.hashtable.PyObjectHashTable.get_item()  
  
File pandas/_libs/hashtable_class_helper.pxi:5753, in pandas._libs.hashtable.PyObjectHashTable.get_item()
```

KeyError: 'tag'

The above exception was the direct cause of the following exception:

Tracebacks from packages

```
KeyError                                Traceback (most recent call last)
Cell In[5], line 9
      5 products = pd.DataFrame({"ID": ["ABC1", "ABC2", "ABC3"],
      6                           "price": [29.99, 39.95, 51.25]})
      8 # Try to access the non-existent "tag" column
----> 9 products["tag"]
```

Tracebacks from packages

```
File /usr/local/lib/python3.8/dist-packages/pandas/core/frame.py:3804 in DataFrame.__getitem__(self, key)
  3802 if self.columns.nlevels > 1:
  3803     return self._getitem_multilevel(key)
-> 3804 indexer = self.columns.get_loc(key)
  3805 if is_integer(indexer):
  3806     indexer = [indexer]

File /usr/local/lib/python3.8/dist-packages/pandas/core/indexes/base.py:3805, in Index.get_loc(self, key, method, tolerance)
  3803     return self._engine.get_loc(casted_key)
  3804 except KeyError as err:
-> 3805     raise KeyError(key) from err
  3806 except TypeError:
  3807     # If we have a listlike key, _check_indexing_error will raise
  3808     # InvalidIndexError. Otherwise we fall through and re-raise
  3809     # the TypeError.
  3810     self._check_indexing_error(key)

KeyError: 'tag'
```

Tracebacks from packages

```
File /usr/local/lib/python3.8/dist-packages/pandas/core/frame.py:3804, in DataFrame.__getitem__(self, key)
    3802 if self.columns.nlevels > 1:
    3803     return self._getitem_multilevel(key)
-> 3804 indexer = self.columns.get_loc(key)
    3805 if is_integer(indexer):
    3806     indexer = [indexer]

File /usr/local/lib/python3.8/dist-packages/pandas/core/indexes/base.py:3805, in Index.get_loc(self, key, method, tolerance)
    3803     return self._engine.get_loc(casted_key)
    3804 except KeyError as err:
-> 3805     raise KeyError(key) from err
    3806 except TypeError:
    3807     # If we have a listlike key, _check_indexing_error will raise
    3808     # InvalidIndexError. Otherwise we fall through and re-raise
    3809     # the TypeError.
    3810     self._check_indexing_error(key)

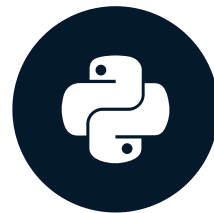
KeyError: 'tag'
```

Let's practice!

INTERMEDIATE PYTHON FOR DEVELOPERS

Error handling

INTERMEDIATE PYTHON FOR DEVELOPERS



Jasmin Ludolf

Senior Data Science Content Developer

Pandas traceback

```
File /usr/local/lib/python3.8/dist-packages/pandas/core/indexes/base.py:3805, in Index.get_loc(self, key, method, tolerance)
    3803     return self._engine.get_loc(casted_key)
    3804 except KeyError as err:
-> 3805     raise KeyError(key) from err
    3806 except TypeError:
    3807     # If we have a listlike key, _check_indexing_error will raise
    3808     # InvalidIndexError. Otherwise we fall through and re-raise
    3809     # the TypeError.
    3810     self._check_indexing_error(key)

KeyError: 'tag'
```

- `except` , `raise`
- Try to anticipate how errors might occur

Design-thinking

- How might people use our custom function?
- Test different approaches
- Find what errors might occur



Error handling in custom functions

```
def average(values):  
    # Calculate the average  
    average_value = sum(values) / len(values)  
    return average_value
```


Where might they go wrong?

- `average()` expects a list or set
- Provide more than one argument ☐
- Use the wrong data type ☐

Where might they go wrong?

```
sales_dict = {"cust_id": ["JL93", "MT12", "IY64"],
              "order_value": [43.21, 68.70, 82.19]}
print(average(sales_dict))
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[5], line 4
      1 sales_dict = {"cust_id": ["JL93", "MT12", "IY64"],
      2                  "order_value": [43.21, 68.70, 82.19]}
----> 4 average(sales_dict)

Cell In[4], line 3, in average(values)
      1 def average(values):
      2     # Calculate the average
----> 3     average_value = sum(values) / len(values)
      4     return average_value

TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Previous error-handling

- Control flow `if` , `elif` , `else`
- Docstrings

try-except

```
def average(values):  
    try:  
        # Code that might cause an error  
        average_value = sum(values) / len(values)  
        return average_value  
    except:  
        # Code to run if an error occurs  
        print("average() accepts a list or set. Please provide a correct data type.")
```

raise

```
def average(values):  
    # Check data type  
    if type(values) in (list, set):  
        # Run if appropriate data type was used  
        average_value = sum(values) / len(values)  
        return average_value
```

raise

```
def average(values):  
    # Check data type  
    if type(values) in (list, set):  
        # Run if appropriate data type was used  
        average_value = sum(values) / len(values)  
        return average_value  
    else:  
        # Run if an Exception occurs  
        raise
```

raise TypeError

```
def average(values):  
    # Check data type  
    if type(values) in (list, set):  
        # Run if appropriate data type was used  
        average_value = sum(values) / len(values)  
        return average_value  
    else:  
        # Run if an Exception occurs  
        raise TypeError("average() accepts a list or set, please provide a correct data type.")
```


raise TypeError output

```
print(average(sales_dict))
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[19], line 11  
      7     else:  
      8         # Run if an Exception occurs  
      9         raise TypeError("average() accepts a list or set, please provide a correct data type.")  
--> 11 average(sales_dict)  
  
Cell In[19], line 9, in average(values)  
      6     return average_value  
      7 else:  
      8     # Run if an Exception occurs  
----> 9     raise TypeError("average() accepts a list or set, please provide a correct data type.")  
  
TypeError: average() accepts a list or set, please provide a correct data type.
```


try-except vs. raise

try - except

- Avoid errors being produced
- Still execute subsequent code

raise

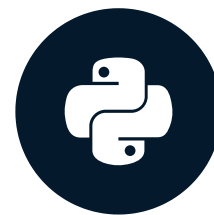
- Will produce an error
- Avoid executing subsequent code

Let's practice!

INTERMEDIATE PYTHON FOR DEVELOPERS

Congratulations

INTERMEDIATE PYTHON FOR DEVELOPERS



Jasmin Ludolf

Senior Data Science Content Developer

Chapter 1 recap

- Built-in functions:

- `print()` , `help()` , `type()`
- `max()` , `min()` , `sum()`
- `len()` , `round()` , `sorted()`

- Modules:

- `string` , `os`

- Packages:

- `pandas`

Chapter 2 recap

```
# Create a custom function
def average(values):
    # Calculate the average
    average_value = sum(values) / len(values)

    # Round the results
    rounded_average = round(average_value, 2)

    # Return rounded_average as an output
    return rounded_average
```

Chapter 2 recap

```
# Create a custom function
def average(values, rounded=False):
    # Round average to two decimal places if rounded is True
    if rounded == True:
        average_value = sum(values) / len(values)
        rounded_average = round(average_value, 2)
        return rounded_average
    # Otherwise, don't round
    else:
        average_value = sum(values) / len(values)
        return average_value
```

Chapter 2 recap

```
def average(values):  
    """  
    Find the mean in a sequence of values and round to two decimal places.  
  
    Args:  
        values (list): A list of numeric values.  
  
    Returns:  
        rounded_average (float): The mean of values, rounded to two decimal places.  
    """  
    average_value = sum(values) / len(values)  
    rounded_average = round(average_value, 2)  
    return rounded_average
```


Chapter 2 recap

```
# Use arbitrary positional arguments
```

```
def average(*args):  
    average_value = sum(args) / len(args)  
    rounded_average = round(average_value, 2)  
    return rounded_average
```

```
# Use arbitrary keyword arguments
```

```
def average(**kwargs):  
    average_value = sum(kwargs.values()) / len(kwargs.values())  
    rounded_average = round(average_value, 2)  
    return rounded_average
```


Chapter 3 recap

- `lambda arguments: expression`

```
names = ["john", "sally", "leah"]

# Apply a lambda function inside map()
capitalize = map(lambda x: x.capitalize(), names)

# Convert to a list
print(list(capitalize))
```

```
['John', 'Sally', 'Leah']
```

Chapter 3 recap

```
-----  
ValueError                                Traceback (most recent call last)  
Cell In[2], line 1  
----> 1 float("Hello")  
  
ValueError: could not convert string to float: 'Hello'
```

- try
- except
- raise

Next steps

- More built-in functions:
 - `zip()`
 - `input()`
 - `reduce()`
 - `filter()`
- More packages and modules:
 - `time`
 - `venv`
 - `requests`
 - `fastapi`

INTERACTIVE COURSE

Introduction to Object-Oriented Programming in Python

[Start](#) [Bookmark](#) [...](#)

 Intermediate  3 hr  11 videos  37 Exercises  16,781 [3100 XP](#)

 Updated: Jul 2025

- Crucial for building software at scale

Congratulations!

INTERMEDIATE PYTHON FOR DEVELOPERS